



Master 2 Intelligence Artificielle Examen Outils de Versioning

Projet : Bibliothèque Numérique avec Système de
Recommandation

Durée : 3 semaines — du 24 Avril 2026 au 15 Mai 2026

Contexte général

Le **Dakar Institute of Technology (DIT)** souhaite développer une plateforme complète de gestion de bibliothèque académique.

Actuellement, la gestion des livres et des emprunts est réalisée de manière **manuelle**, ce qui entraîne :

- Difficulté de suivi des livres
- Absence de statistiques fiables
- Gestion inefficace des emprunts
- Manque d'accès numérique pour les étudiants

La direction souhaite une **plateforme web moderne** avec :

- Gestion des livres, utilisateurs et emprunts
- **Système de recommandation de livres** basé sur l'historique des emprunts
- Versioning avancé du code, des données et du modèle

Vous êtes chargé de développer l'application complète et de mettre en place une gestion avancée du versioning.

Objectifs pédagogiques

Ce projet vise à évaluer votre capacité à :

- Développer une application **microservices** complète
- Implémenter un **système de recommandation** (ML)
- Utiliser les fonctionnalités **avancées de Git**
- Conteneuriser l'application avec **Docker**
- Versionner les **données et le modèle ML** avec **DVC**

- Déployer avec **Docker Compose**

Architecture du projet

L'application complète doit respecter l'architecture suivante :

- Architecture **microservices** (5 services minimum)
- Communication via **API REST**
- Base de données relationnelle
- Conteneurisation avec Docker
- Orchestration avec Docker Compose
- Versioning code : Git
- Versioning données/modèle : DVC

Technologies autorisées

Les étudiants sont libres de choisir parmi les technologies suivantes :

Backend (autres services)

- Spring Boot
- Node.js (Express)
- Django / Django REST
- Flask
- Scikit-learn (SVD ou KNN)

Frontend

- React
- Angular
- Vue.js
- HTML / CSS / JavaScript

Base de données

- MySQL
- PostgreSQL

Versioning

- Git (avancé)
- DVC (Data Version Control)
- Remote DVC : Google Drive / S3 / Dropbox

Microservices à implémenter

1 — Service Livres

Fonctionnalités :

- Ajouter un livre
- Modifier un livre
- Supprimer un livre
- Lister les livres
- Recherche par titre, auteur ou ISBN

2 — Service Utilisateurs

- Création d'utilisateurs
- Liste des utilisateurs
- Gestion des types (Étudiant, Professeur, Personnel)
- Consultation du profil utilisateur

3 — Service Emprunts

- Emprunter un livre
- Retourner un livre
- Historique des emprunts
- Détection des retards
- Export des données d'historique pour le ML

4 — Service Recommandation

- API REST (FastAPI) qui expose des recommandations
- Modèle ML entraîné sur l'historique des emprunts
- Endpoint : GET /recommendations/{user_id} → liste de livres recommandés
- Endpoint : POST /train → ré-entraînement du modèle

Partie 1 : Développement de l'application

1.1 Services : Livres, Utilisateurs, Emprunts

Implémentez les 3 services de base (Livres, Utilisateurs, Emprunts) avec :

- Au moins 5 endpoints par service
- Communication REST entre services
- Persistance des données

1.2 Service de recommandation

Développez l'API FastAPI avec :

- Chargement du modèle pré-entraîné
- Endpoint de recommandation
- Endpoint de ré-entraînement
- Gestion des erreurs

1.3 Frontend

Interface utilisateur permettant :

- Consultation du catalogue
- Gestion des emprunts
- Affichage des recommandations personnalisées

Partie 1 : Docker & Docker Compose

1.1 Dockerfiles

Écrivez un Dockerfile pour chaque service, avec :

- Multi-stage pour l'API recommandation
- Utilisation d'utilisateur non-root
- Exposition des ports appropriés

1.2 Docker Compose

Créez un `docker-compose.yml` avec :

- Tous les services (db, livres, utilisateurs, emprunts, reco-api, frontend)
- Profil `dev` (hot-reload)
- Profil `prod` (build)
- Réseau dédié

1.3 Volumes

Configurez les volumes pour :

- Persistance de la base de données
- Stockage du modèle ML
- Données d'entraînement

Partie 2 : DVC - Data Version Control

2.1 Initialisation

Initialisez DVC et configurez un remote sur Google Drive.

2.2 Dataset

Exportez l'historique des emprunts depuis PostgreSQL vers `loans.csv` et versionnez-le avec DVC.

2.3 Pipeline DVC

Créez un pipeline avec 3 étapes dans `dvc.yaml` :

1. `preprocess.py` : nettoie les données → `loans_clean.csv`
2. `train.py` : entraîne un modèle (SVD/KNN) → `model.pkl`
3. `evaluate.py` : calcule RMSE, MAE → `metrics.json`

2.4 Reproduction

Exécutez `dvc repro` et expliquez le rôle de `dvc.lock`.

2.5 Métriques

Affichez les métriques avec `dvc metrics show` et comparez deux versions du modèle.

4.6 Versioning du modèle

Versionnez `model.pkl` avec DVC et montrez comment l'API charge la bonne version.

Livrables attendus

1 — Code source

Dépôt **GitHub** contenant :

- Code complet de tous les microservices
- Dockerfile de chaque service
- `docker-compose.yml`
- Scripts Python pour DVC (`preprocess`, `train`, `evaluate`)
- `dvc.yaml`, `dvc.lock`
- Fichiers `.dvc` pour les données et le modèle
- Jenkinsfile (bonus)

2 — Rapport

Document PDF contenant :

- Présentation du projet
- Architecture du système (schéma)
- Description des APIs
- Workflow Git utilisé
- Pipeline DVC (schéma des dépendances)
- Métriques du modèle
- Captures d'écran de l'application

3 — README

Le README doit expliquer :

- Installation et lancement avec Docker Compose
- Initialisation de la base de données
- Entraînement et reproduction du modèle avec DVC
- Tests des endpoints

Barème

Partie	Points
Développement application (services + reco + frontend)	6
Git Avancé	4
Docker & Docker Compose	4
DVC	6
Total	20

Bonus (+2 points) : Pipeline CI/CD avec Jenkins ou GitHub Actions.